GSoC21 W2: LFortran Unraveling

Rohit Goswami · 2021-06-18 · gsoc21 · fortran · ramblings

> *Digging into language standards and back-ends for* `lfortran`

Background

As discussed in a previous post in this series, I have been spending roughly half of each working day with LFortran as part of the 2021 Google Summer of Code under the fortran-lang organization, mentored by Ondrej Certik.

Logistics

Some of the meeting points are to be expanded on below.

- Met with Ondrej on Tuesday, as discussed previously
  - Talked about language server implementations
    - Looked into `rtags` and generating a compilation-database
    - Discussed how the C++ concept of having file based units makes this simpler than the Fortran form, which recognizes no file based program units
- Talked about the status of the different back-ends
- Discussed LLVM and MLIR, in the context of Flang (the `f18` compiler)
  - Also briefly touched upon `legacy-flang` and historical issues
- Discussed standardization of the mod-files
  - Decided this is not a good idea, because a lot of build systems expect Fortran compilers to generate `.mod` files, even if they are completely incompatible
  - The standard does not specify what should be contained in a `mod` file
  - Every `mod` file of every compiler is unique and needs to be handled separately
  - The goal (of this project too) is to handle at the very least conversion of `gfortran` module files to `lfortran` module files

Overview

For the second week of my project on getting `lfortran` to compile `dftatom` and in general form a usable compiler ecosystem to facilitate greater adoption in the community, I opted to take a bit of a step back and dig into the historical evolution of both the Fortran standard itself Lyon (1980) and also the compiler ecosystem. This was also in no small part due to the fact that I ended up moving during this week, which forced me to spend a lot of time cleaning and playing with adult LEGO [^fn:1]. This naturally left me with plenty of time to both participate [^fn:2] in the monthly Fortran-lang call (which was rather explosive [^fn:3]) and also contemplate the overall ecosystem of the standards committee and compilers. Many more issues were set up and will be completed over the weekend.

Merge Requests

Minor installation bug (985)

Fixed a small issue with the `cmake` files

Assigned Issues

Handling `kind` in the ASR (357)

A straightforward correction to make the ASR handle more common, but non-standard use cases

GFortran Module v15 support (355)

Currently only supports v14

Runtime Math and the CPP back-end (354)

Might take a little longer, does not compile at the moment

Additional Tasks

I intend to write more about the Fortran standard and go through more of the compiler code I can get my hands on, since that should give me a better handle on the different way the standards have been implemented (and augmented!). As part of this, I'll probably handle some of issue 350; regarding the main `lfortran.org` site.

LFortran and Back-ends

LLVM

This is the default, and the most performance oriented. It is however, slightly more complicated to extend due to the intricacies of the LLVM syntax.

C++

This is a newer back-end, perfect for rapid prototyping; and can be selected by passing the `--backend=cpp` flag to `lfortran` (e.g. with `FFLAGS`). This is easier to work with in that many features boil down to `stdlib` calls. However, this back-end requires KOKKOS. It also has uglier error handling as shown in Fig. 1. Since this is more attractive as a candidate for the math intrinsics as a first approximation, it is of considerable interest to me.

It is however, still fairly straightforward to work with.

```
cd $LFROOT # LFortran root
export LFORTRAN_KOKKOS_DIR=$LFROOT/ext/kokkos
mkdir extsrc && cd extsrc
gh repo clone kokkos/kokkos
cd kokkos && mkdir build && cd build
cmake -DCMAKE_INSTALL_PREFIX=$LFORTRAN_KOKKOS_DIR -DKokkos_ARCH_HSW=On ..
make -j$(nproc)
make install
cd $LFORTRAN_KOKKOS_DIR # Need to make a symlink
ln -sf lib64/ lib
cd ../../examples/project1
FC=lfortran FFLAGS="--backend=cpp" cmake .
make
./project1 # Profit
```

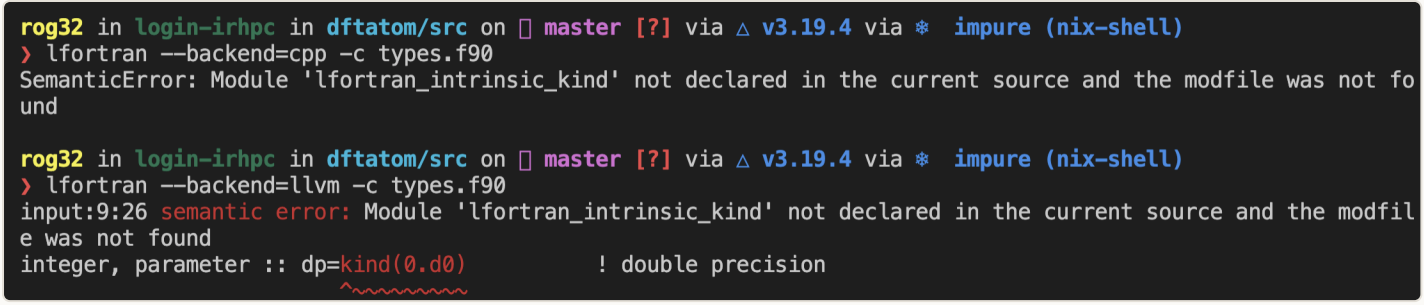

Figure 1: Contrasting backends

Inspecting intermediates

The module files cannot currently be read or queried from `lfortran`, however a representation of what gets compressed into these files can be obtained with the `--show-asr` flag. Additionally, a very `python` inspired `--show-stacktrace` is implemented to help narrow down areas which need to be augmented.

gfortran

```
cd $DFTATM/src
gfortran -c types.f90
# Read gfortran module
zcat types.mod
# Output
GFORTRAN module version '15' created from types.f90
(() () () () () () () () () () () () () () () () () () () () () ()
() () ())

()

()

()

()

()

(2 'dp' 'types' '' 1 ((PARAMETER UNKNOWN-INTENT UNKNOWN-PROC UNKNOWN
IMPLICIT-SAVE 0 0) () (INTEGER 4 0 0 0 INTEGER ()) 0 0 () (CONSTANT (
INTEGER 4 0 0 0 INTEGER ()) 0 '8' ()) () 0 () () () 0 0)
)

('dp' 0 2)
```

lfortran

```
cd $DFTATM/src
lfortran --show-asr -c types.f90
# Output
(TranslationUnit (SymbolTable 1 {lfortran_intrinsic_kind: (Module (SymbolTable 4 {dkind: (Function
(SymbolTable 5 {r: (Variable 5 r ReturnVar () Default (Integer 4 []) Source Public), x: (Variable 5 x In ()
Default (Real 8 []) Source Public)}) dkind [(Var 5 x)] [(= (Var 5 r) (ConstantInteger 8 (Integer 4 [])))] (Var
5 r) Intrinsic Public Implementation), kind: (Function (SymbolTable 6 {r: (Variable 6 r ReturnVar () Default
(Integer 4 []) Source Public), x: (Variable 6 x In () Default (Logical 4 []) Source Public)}) kind [(Var 6 x)]
[(= (Var 6 r) (ConstantInteger 4 (Integer 4 [])))] (Var 6 r) Intrinsic Public Implementation), lkind:
(Function (SymbolTable 7 {r: (Variable 7 r ReturnVar () Default (Integer 4 []) Source Public), x: (Variable 7
x In () Default (Logical 4 []) Source Public)}) lkind [(Var 7 x)] [(= (Var 7 r) (ConstantInteger 4 (Integer 4
[])))] (Var 7 r) Intrinsic Public Implementation), selected_int_kind: (Function (SymbolTable 8 {R: (Variable 8
R In () Default (Integer 4 []) Source Public), res: (Variable 8 res ReturnVar () Default (Integer 4 []) Source
Public)}) selected_int_kind [(Var 8 R)] [(If (Compare (Var 8 R) Lt (ConstantInteger 10 (Integer 4 []))
(Logical 4 [])) [(= (Var 8 res) (ConstantInteger 4 (Integer 4 [])))] [(= (Var 8 res) (ConstantInteger 8
(Integer 4 [])))])] (Var 8 res) Intrinsic Public Implementation), selected_real_kind: (Function (SymbolTable 9
{R: (Variable 9 R In () Default (Integer 4 []) Source Public), res: (Variable 9 res ReturnVar () Default
(Integer 4 []) Source Public)}) selected_real_kind [(Var 9 R)] [(If (Compare (Var 9 R) Lt (ConstantInteger 7
(Integer 4 [])) (Logical 4 [])) [(= (Var 9 res) (ConstantInteger 4 (Integer 4 [])))] [(= (Var 9 res)
(ConstantInteger 8 (Integer 4 [])))])] (Var 9 res) Intrinsic Public Implementation), skind: (Function
(SymbolTable 10 {r: (Variable 10 r ReturnVar () Default (Integer 4 []) Source Public), x: (Variable 10 x In ()
Default (Real 4 []) Source Public)}) skind [(Var 10 x)] [(= (Var 10 r) (ConstantInteger 4 (Integer 4 [])))]
(Var 10 r) Intrinsic Public Implementation)}) lfortran_intrinsic_kind [] .true.), types: (Module (SymbolTable
2 {dp: (Variable 2 dp Local (FunctionCall 2 kind () [(ConstantReal "0.d0" (Real 4 []))] [] (Integer 4 []))
Parameter (Integer 4 []) Source Public), kind: (ExternalSymbol 2 kind 4 kind lfortran_intrinsic_kind kind
Private)}) types [lfortran_intrinsic_kind] .false.)}) [])
```

Note that, `lfortran` is expected to eventually interoperate with existing `mod` files from compilers like `gfortran`.

```
lfortran mod --show-asr types.mod
# Output
Traceback (most recent call last):
  File "/build/glibc-2.32/csu/../sysdeps/x86_64/start.S", line 120, in _start()
  Binary file "/nix/store/hp8wcylqr14hrrpqap4wdrwzq092wfln-glibc-2.32-37/lib/libc.so.6", in
__libc_start_main()
  File "$HOME/Git/Github/Fortran/mylf/src/bin/lfortran.cpp", line 1076, in ??
    asr = LFortran::mod_to_asr(al, arg_mod_file);
  File "$HOME/Git/Github/Fortran/mylf/src/lfortran/mod_to_asr.cpp", line 361, in
LFortran::mod_to_asr(Allocator&, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char>
>)
    return parse_gfortran_mod_file(al, s);
LFortranException: Only GFortran module version 14 is implemented so far
```

Which is the source of an issue assigned to me.

Conclusions

As I cross my fingers and hope my tenuously tethered wifi is equal to the task of uploading this post; I can only hope that the installation of my WiFi on Monday goes smoothly enough to be restored to full working capacity for the next week. Until then, I shall trawl the code-bases and standards for an inkling of what makes compiler design the art form as described in Cooper and Torczon (2011).

References

Cooper, Keith, and Linda Torczon. 2011. *Engineering a Compiler*. Elsevier. <https://books.google.com?id=_tgh4bgQ6PAC>.

Lyon, G. E. 1980. *Using Ans Fortran*. National Bureau of Standards. <https://books.google.com?id=8ymHAQAACAAJ>.

```
counterpart Rumfatalagerinn is that some people enjoy assembling
their furniture the same way kids enjoy LEGO
```

```
text-only participation
```

```
committee being far too conservative, though many of the concerns
reflect on the dearth of community efforts compared to other
languages; Fortran is not after-all coupled to a compiler unlike
modern languages like Rust and Julia
```